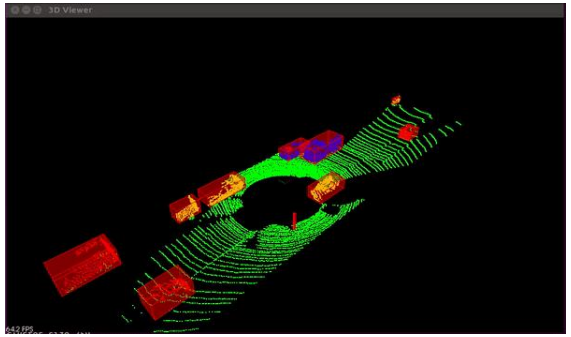
# Sensors and object clustering

Nacho Sañudo
HIPERT - University of Modena and Reggio Emilia
isanudo@unimore.it

Academic year: 2026/2027

# Euclidean clustering detection



- **Objective**: Find and segment the individual object point clusters lying on the plane
  1. Down sample the point cloud
  2. Segment the ground plane
  3. Create a KD-tree representation for the input point cloud dataset
  4. Compute Euclidean distance to build the cluster list
  5. The algorithm terminates when all points have been processed and are now part of the list of point clusters

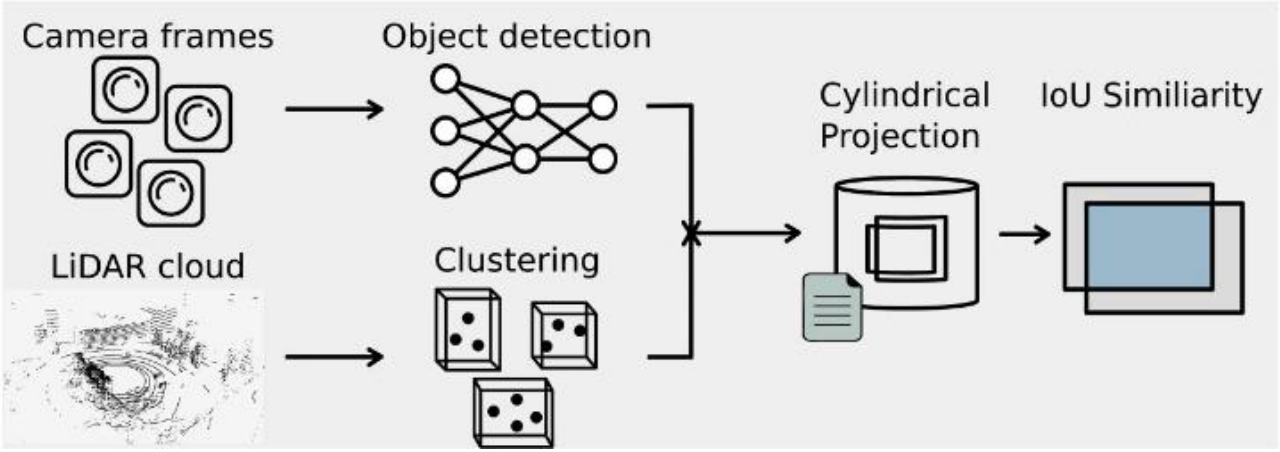
```
sudo apt-get install pcl-tools
sudo apt install libpcl-dev
```

# Perception HiPeRT SW stack



- LiDAR and camera detection (sensor fusion)
  - Clustering - LiDAR (VLP-32C) –DBScan / **Euclidean clustering**
  - Object detection - Camera (4 sekonix/120ºFOV) - YOLO

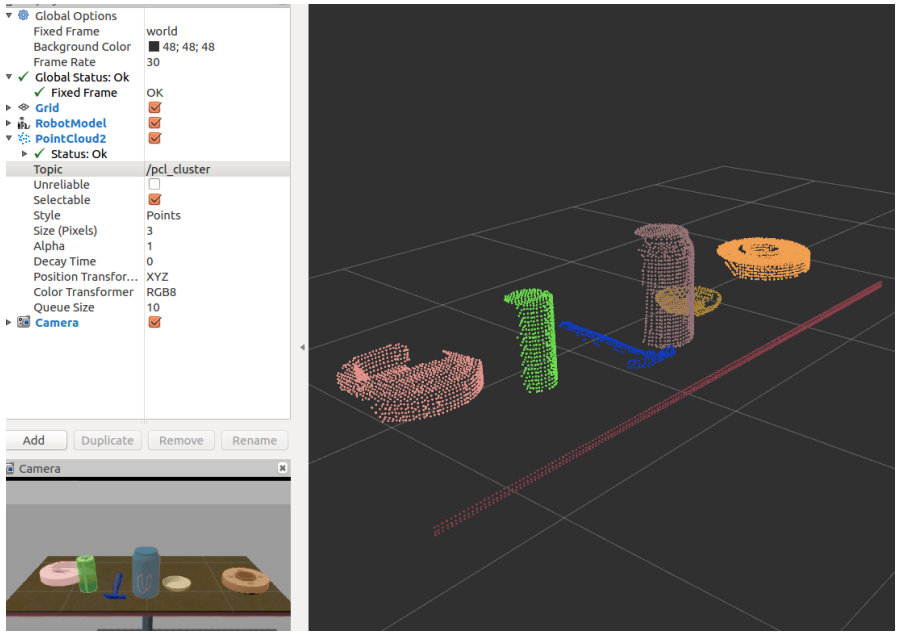
https://www.youtube.com/watch?v=qkWqRSgUFmw




M. Verucchi, L. Bartoli, F. Bagni, F. Gatti, P. Burgio and M. Bertogna, "Real-Time clustering and LiDAR-camera fusion on embedded platforms for self-driving cars", in proceedings in IEEE Robotic Computing (2020)

# Point cloud exercise

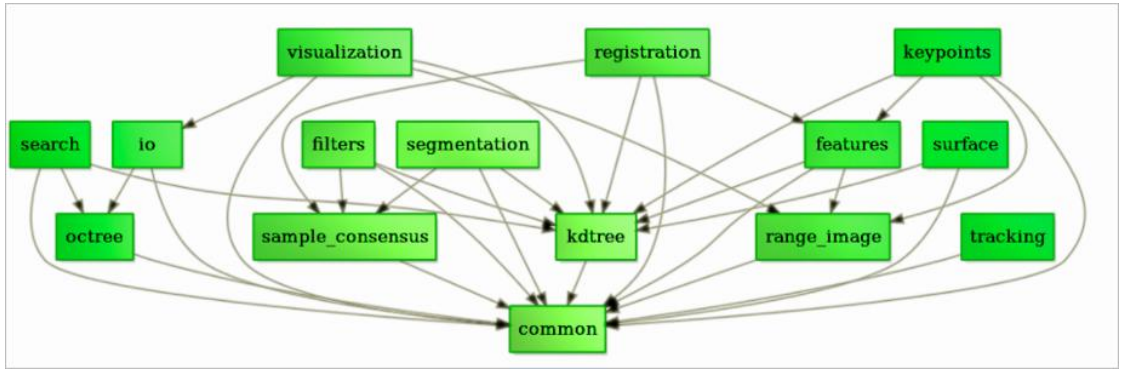
- **Objective**: Find and segment the individual object point clusters lying on the plane (**Euclidean clustering**)
- A clustering method is used to organize an unorganized point cloud into smaller parts
  - identify which points in a point cloud belong to the same object
    1. Down sample the point cloud
    2. Segment the ground plane
    3. Create a KD-tree representation for the input point cloud dataset
    4. Compute Euclidean distance to build the cluster list
    5. the algorithm terminates when all points have been processed and are now part of the list of point clusters

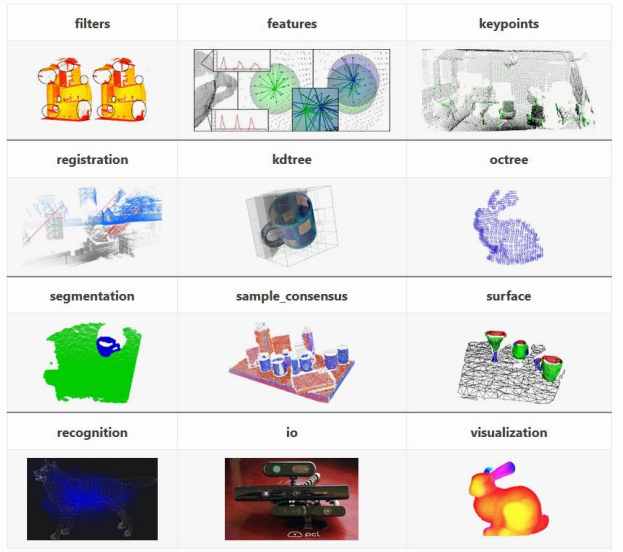



Source: https://pcl.readthedocs.io/projects/tutorials/

# Point cloud library

› The Point Cloud Library (PCL) is a standalone, large scale, open project for 2D/3D image and point cloud processing. PCL is released under the terms of the **BSD license**, and thus free for commercial and research use




Source: https://pointclouds.org/

# Point cloud

- A point cloud is a set of data points used to represent a collection of multi dimensional points
- In a 3D point cloud, the points usually represent the X, Y, and Z coordinates of a sampled object
- Point clouds can be created using a LiDAR or a stereo camera, RGB-D, Stereo cameras



# PCD data

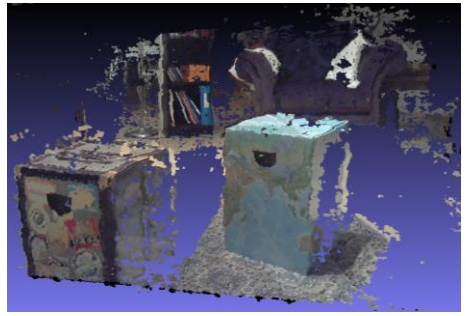
- PCD (Point Cloud Data) is a file format that supports 3D point cloud data
  - **FIELDS** - specifies the name of each dimension/field that a point can have
  - **SIZE** - specifies the size of each dimension in bytes
  - **WIDTH** & **HEIGHT**– determines the number of points of the cloud in WIDTH and HEIGHT
    - Image-like organized structure, with 480 rows and 640 columns
    - cloud.width = 640;
    - cloud.height = 480;
    - thus 640*480=307200 points total in the dataset
  - **POINTS** - specifies the total number of points in the cloud
  - **DATA** - specifies the data type that the point cloud data is stored in (ascii or binary)

```
# .PCD v.7 - Point Cloud Data file format
VERSION .7
FIELDS x y z rgb
SIZE 4 4 4 4
TYPE F F F F
COUNT 1 1 1 1
WIDTH 213
HEIGHT 1
VIEWPOINT 0 0 0 1 0 0 0
POINTS 213
DATA ascii
0.93773 0.33763 0 4.2108e+06
0.90805 0.35641 0 4.2108e+06
0.81915 0.32 0 4.2108e+06
0.97192 0.278 0 4.2108e+06
0.944 0.29474 0 4.2108e+06
0.98111 0.24247 0 4.2108e+06
```

# Point cloud types

- PointXYZ - float x, y, z
- PointXYZI - float x, y, z, intensity
- PointXYZRGB - float x, y, z, rgb (RGBA information packed into a single integer)
- PointXYZRGBA - float x, y, z, uint32 t rgbaNormal - float normal[3], curvature (RGB information packed into a float)





# Point cloud visualization

## pcl_viewer

```
The viewer window provides interactive commands; for help, press 'h' or 'H' from within the window.
Syntax is: pcl_viewer <file_name 1..N>.<pcd or vtk> <options>
  where options are:
                     -bc r,g,b                = background color
                     -fc r,g,b                = foreground color
                     -ps X                    = point size (1..64)
                     -opaque X                = rendered point cloud opacity (0..1)
                     -shading X               = rendered surface shading ('flat' (default), 'gouraud', 'phong')
                     -position x,y,z          = absolute point cloud position in metres
                     -orientation r,p,y       = absolute point cloud orientation (roll, pitch, yaw) in radians
                     -ax n                    = enable on-screen display of XYZ axes and scale them to n
                     -ax_pos X,Y,Z            = if axes are enabled, set their X,Y,Z position in space (default 0,0,0)

                     -cam (*)                 = use given camera settings as initial view
 (*) [Clipping Range / Focal Point / Position / ViewUp / Distance / Field of View Y / Window Size / Window Pos] or use a <filename.cam> that contains the same information.

                     -multiview 0/1           = enable/disable auto-multi viewport rendering (default disabled)


                     -normals 0/X             = disable/enable the display of every Xth point's surface normal as lines (default disabled)
                     -normals_scale X         = resize the normal unit vector size to X (default 0.02)

                     -pc 0/X                  = disable/enable the display of every Xth point's principal curvatures as lines (default disabled)
                     -pc_scale X              = resize the principal curvatures vectors size to X (default 0.02)

                     -immediate_rendering 0/1 = use immediate mode rendering to draw the data (default: disabled)
                                                Note: the use of immediate rendering will enable the visualization of larger datasets at the expense of extra RAM.
                                                See http://en.wikipedia.org/wiki/Immediate_mode for more information.
                     -vbo_rendering 0/1       = use OpenGL 1.4+ Vertex Buffer Objects for rendering (default: disabled)
                                                Note: the use of VBOs will enable the visualization of larger datasets at the expense of extra RAM.
                                                See http://en.wikipedia.org/wiki/Vertex_Buffer_Object for more information.

                     -use_point_picking       = enable the usage of picking points on screen (default disabled)

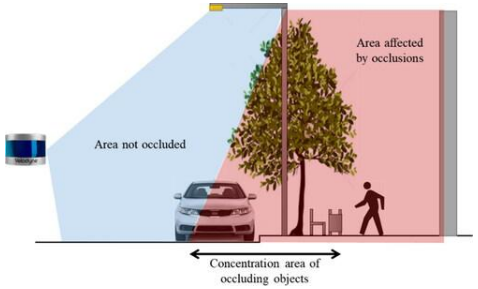
                     -optimal_label_colors    = maps existing labels to the optimal sequential glasbey colors, label_ids will not be mapped to fixed colors (default disabled)


(Note: for multiple .pcd files, provide multiple -{fc,ps,opaque,position,orientation} parameters; they will be automatically assigned to the right file)
```

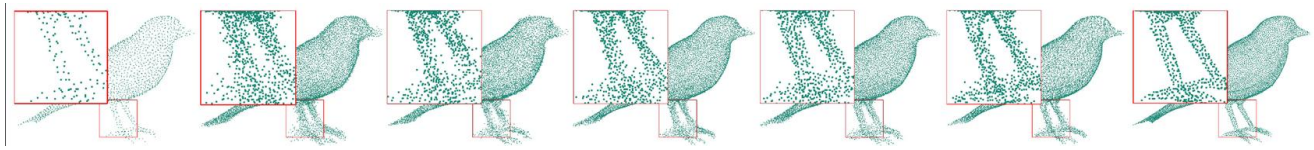
# Basic functions

- pcl::PointCloud<pcl::PointXYZ>::Ptr cloud
  - pcl::PointCloud< PointT > name () → default constructor
- pcl::io::loadPCDFile("path", *cloud);
  - Load a PCD file into a templated PointCloud type.
- pcl::visualization::CloudViewer viewer("id");
  - Construct a cloud viewer, with a window name.
- viewer.showCloud(cloud);
  - Show a cloud, with an optional key for multiple clouds.
  - void pcl::visualization::CloudViewer::showCloud ( const ColorCloud::ConstPtr & cloud, const std::string & cloudname = "cloud")

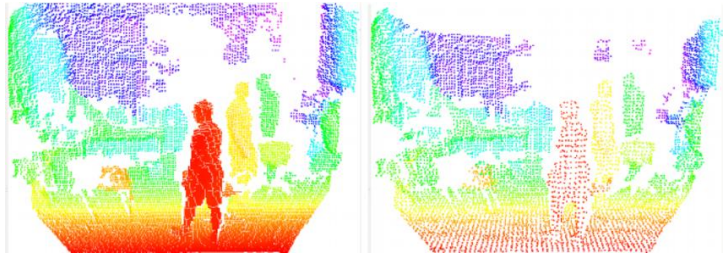
# LiDAR point cloud



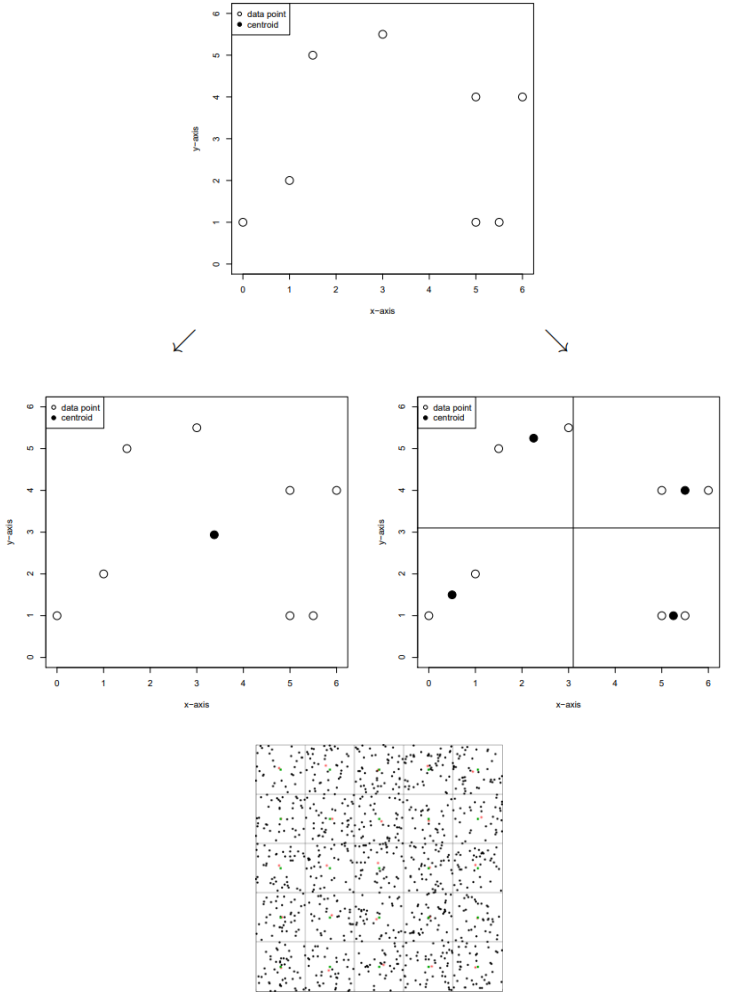

| Manufacturer & Model | Field of View (FoV) (H × V) | Range (10% reflectivity) | Frame Rate (Frequency) | Point Cloud Output Rate | Estimated Data Rate (MB/s)* |
|---|---|---|---|---|---|
| Hesai AT128 / AT128P | 120° × 25.6° | 200 m | 10 Hz | ~1.53M – 2.56M pts/sec | ~30 – 51 MB/s |
| InnovizTwo | Up to 125° × 40° | 220 m | 10 Hz – 20 Hz | ~1.5M – 2.5M pts/sec | ~30 – 50 MB/s |
| Luminar Iris / Iris+ | 120° × 28° | 250 m – 300 m | 10 Hz (Variable 1–30 Hz) | ~1.5M – 2.0M pts/sec | ~30 – 40 MB/s |
| RoboSense M1 / M1 Plus | 120° × 25° | 150 m – 200 m | 10 Hz – 20 Hz | ~1.5M pts/sec | ~30 MB/s |
| Ouster OS2 (128-ch) | 360° × 22.5° | 200 m | 10 Hz – 20 Hz | ~2.62M pts/sec | ~52 MB/s |
| Hesai Pandar128 | 360° × 40° | 200 m | 10 Hz – 20 Hz | ~3.45M pts/sec | ~69 MB/s |



# Voxel filtering



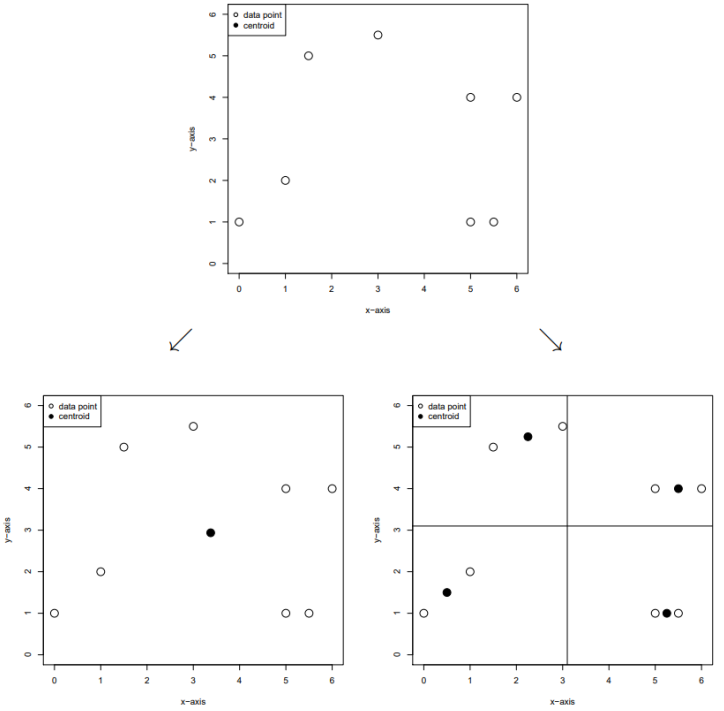
- Point clouds store a lot of detailed information about physical space
  - Not all the information is required
- **Centroid voxel filtering**: technique used to uniformly reduce the number of points (also called downsample)
  - The voxel grid filter down-samples the data by taking a spatial average of the points in the cloud (i.e., considering the centroid)




https://libpointmatcher.readthedocs.io/en/latest/DataPointsFilterDev/

# Voxel filtering

- Black dots are the centroids of each voxel
  - Fig. left side → big voxel size
  - Fig. right side → smaller voxel size
- A voxel grid filter down samples the data by averaging the points for each voxel grid in the point cloud
- **setLeafSize(x,y,z) set the voxel grid leaf size**
  - **The bigger the leaf size the more the points filtered**

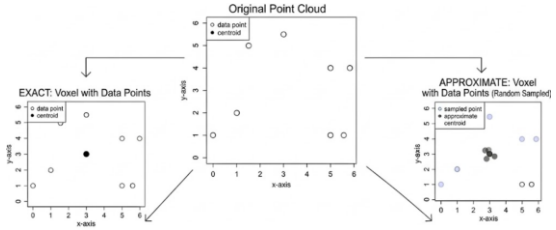



```
pcl::VoxelGrid<pcl::PCLPointCloud2> sor;
sor.setInputCloud (cloud);
sor.setLeafSize (0.01f, 0.01f, 0.01f);
sor.filter (*cloud_filtered);
```

# Voxel filtering

- A pcl::VoxelGrid filter is created, the input data is passed, and the output is computed and stored in cloud_filtered.
- pcl::VoxelGrid<pcl::PointXYZ> sor;
  - Declare a variable called sor using PointXYZ as type
- sor.setInputCloud (cloud);
  - Cloud to be filtered
- sor.setLeafSize (0.01f, 0.01f, 0.01f);
  - Set the voxel grid leaf size → **a leaf size of 1cm**
- sor.filter (*cloud_filtered);
  - void pcl::VoxelGrid< PointT >::filter (PointCloud &  output)
  - Downsample a Point Cloud using a voxelized grid approach

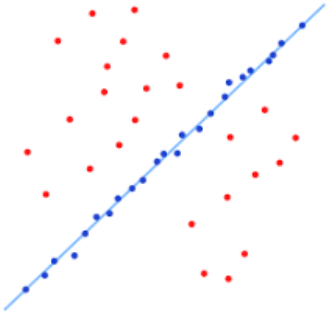
# Alternative - Approximative voxel filtering




- **Approximative voxel filtering**: Instead of computing the centroid of all points in a voxel, the output point is the centroid of the voxel itself
- Although less precise, this method eliminates the need for an additional data scan, optimizing performance
  - Each point is assigned to a voxel
  - If a voxel has no assigned points, its centroid is added to the output cloud
  - For non-spatial attributes (e.g., color, intensity), the output point adopts the values of the first point that activates the voxel
- "This is similar to the VoxelGrid filter. This class works best if points that are stored in memory next to each other (in the input point cloud), are also somewhat close in 3D euclidean space (this is for example usually the case for organized point clouds). If the points have no storage order (e.g. in synthetic, randomly generated point clouds), this class will give very poor results, and VoxelGrid should be used instead."

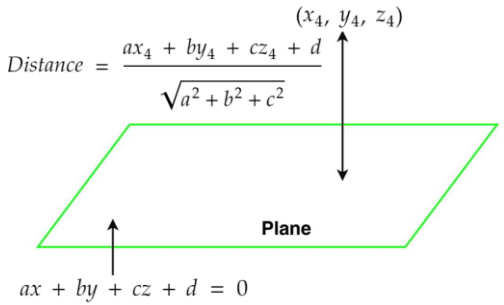
Source: https://pointclouds.org/documentation/classpcl_1_1_approximate_voxel_grid.html

# Ground segmentation - RANSAC



- We need to separate the ground from the other objects
- **Random sample consensus** (**RANSAC**) is an iterative method to estimate parameters of a mathematical model from a set of observed data that contains outliers
  - The algorithm groups the data into inliers and outliers
    - Inliers are the points that represent the table
    - Outliers are the points that represent the objects lying on the table
- 2D Ransac
  - Select two random points
  - Fit a line for those two points
  - Compute the points that are in/close to the line (score)
    - If the distance is close to the defined distance tolerance, we can add the point to the list of inliers
  - Repeat the process with other two random points
  - Choose the model which has the best "score"

# 3D RANSAC




- **Objective**: segment objects (pedestrian, vehicles etc…) that are above the plane
- 3D RANSAC
  - Select three random points
  - Form a plane with these points (plane model)
  - Compute the number of points that fit into the plane model (we consider as **inliers** the points that lies close to the plane)
    - If the points are close close to the defined distance tolerance, we can add the point to the list of inliers
      - how close a point must be to the model in order to be considered an inlier
  - **Repeat** the process N times
  - Select the plane model that "collects" more points

```
// Create the segmentation object for the planar model and set all the parameters
pcl::SACSegmentation<pcl::PointXYZ> seg;
pcl::PointIndices::Ptr inliers (new pcl::PointIndices);
pcl::ModelCoefficients::Ptr coefficients (new pcl::ModelCoefficients);
pcl::PointCloud<pcl::PointXYZ>::Ptr cloud_plane (new pcl::PointCloud<pcl::PointXYZ> ());
pcl::PCDWriter writer;
seg.setOptimizeCoefficients (true);
seg.setModelType (pcl::SACMODEL_PLANE);
seg.setMethodType (pcl::SAC_RANSAC);
seg.setMaxIterations (100);
seg.setDistanceThreshold (0.02);
```

# PCL Ground Segmentation

- pcl::SACSegmentation<pcl::PointXYZ> seg;
  - Class used for performing the segmentation
  - Provides function segment(out1,out2)
    - [out1] inliers: the resultant point indices that support the model found (inliers)
    - [out2] model_coefficients: the resultant model coefficients
- pcl::PointIndices::Ptr inliers (new pcl::PointIndices);
  - Vector that stores the index of the points that represents the plane
- pcl::ModelCoefficients::Ptr coefficients (new pcl::ModelCoefficients);
  - Estimated plane parameters (in ax + by + cz + d = 0 form)
    - coefficients->values[0], coefficients->values[1], coefficients->values[2] , coefficients->values[3]
- pcl::ExtractIndices<pcl::PointXYZ> extract;
  - ExtractIndices extracts a set of indices from a point cloud.

# PCL Ground Segmentation

- Create the :pcl:`SACSegmentation <pcl::SACSegmentation>` object and set the model and method type
  - This is also where we specify the "distance threshold", which determines how close a point must be to the model in order to be considered an inlier
    - We use the RANSAC method (pcl::SAC_RANSAC) as the robust estimator of choice

```
pcl::ModelCoefficients::Ptr coefficients (new pcl::ModelCoeffici
pcl::PointIndices::Ptr inliers (new pcl::PointIndices);
// Create the segmentation object
pcl::SACSegmentation<pcl::PointXYZ> seg;
// Optional
seg.setOptimizeCoefficients (true);
// Mandatory
seg.setModelType (pcl::SACMODEL_PLANE);
seg.setMethodType (pcl::SAC_RANSAC);
seg.setDistanceThreshold (0.01);
seg.setInputCloud (cloud);
seg.segment (*inliers, *coefficients);
```

# Models and estimators

- Available RANSAC models:
  - Plane (pcl::SACMODEL_PLANE): Fits a plane in 3D space
  - Line (pcl::SACMODEL_LINE): Detects linear structures
  - Circle (2D/3D) (pcl::SACMODEL_CIRCLE2D / pcl::SACMODEL_CIRCLE3D): Fits circular shapes in 2D or 3D
  - Sphere (pcl::SACMODEL_SPHERE): Detects spherical objects
  - Cylinder (pcl::SACMODEL_CYLINDER): Fits cylindrical shapes
  - Cone (pcl::SACMODEL_CONE): Detects conical structures
  - Parallel Lines/Planes (pcl::SACMODEL_PARALLEL_LINE / pcl::SACMODEL_PARALLEL_PLANE): Fits multiple parallel lines or planes
- Different available estimators in PCL:
  - RANSAC (pcl::SAC_RANSAC): Standard robust model fitting method
  - LMEDS (pcl::SAC_LMEDS): Minimizes median squared residuals
  - MLESAC (pcl::SAC_MLESAC): Combines RANSAC with maximum likelihood estimation
  - MSAC (pcl::SAC_MSAC): Minimizes error using an M-estimator
  - Randomized RANSAC (pcl::SAC_RRANSAC): Optimized RANSAC with randomized sampling
  - PROSAC (pcl::SAC_PROSAC): Progressive RANSAC prioritizing high-confidence points
  - Parallel RANSAC (pcl::SAC_RANSAC_PARALLEL): Accelerated RANSAC using multi-threading

http://andrewd.ces.clemson.edu/courses/cpsc805/references/nearest_search.pdf

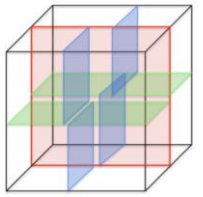
# K-D Tree

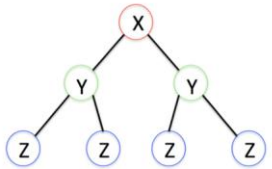
| ***k*-d tree** | |
|---|---|
| **Type** | Multidimensional BST |
| **Invented** | 1975 |
| **Invented by** | Jon Louis Bentley |

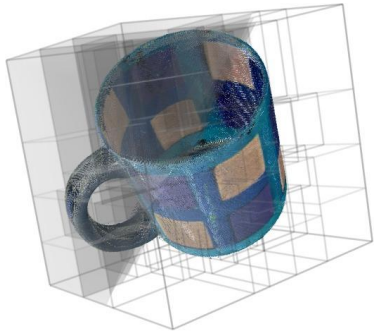
**Time complexity in big O notation**

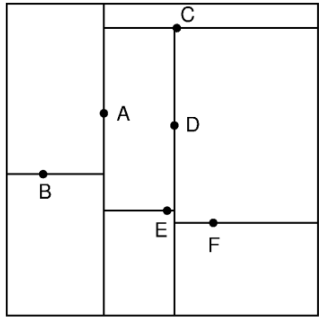
| Algorithm | Average | Worst case |
|---|---|---|
| **Space** | $O(n)$ | $O(n)$ |
| **Search** | $O(\log n)$ | $O(n)$ |
| **Insert** | $O(\log n)$ | $O(n)$ |
| **Delete** | $O(\log n)$ | $O(n)$ |

- How to store the point cloud?
  - Arrays? → complex to compute nearest neighbor
    - O(n), O(log n) if the array is sorted using binary search
  - In computer science, a K-D tree (short for k-dimensional tree) is a space-partitioning data structure for organizing points in a k-dimensional space
    - **Powerful to compute nearest neighbor points**
      - Nearest neighbor searches are an important operation when working with point cloud data and it is used to find correspondences between groups of points (create clusters)
    - Insertion and deletion is faster in KD-Trees than in sorted arrays

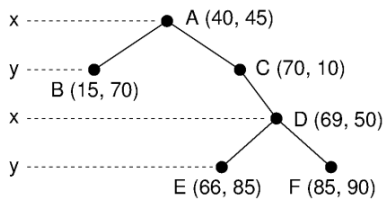







(a)




(b)

x
y
x

# K-D Tree idea

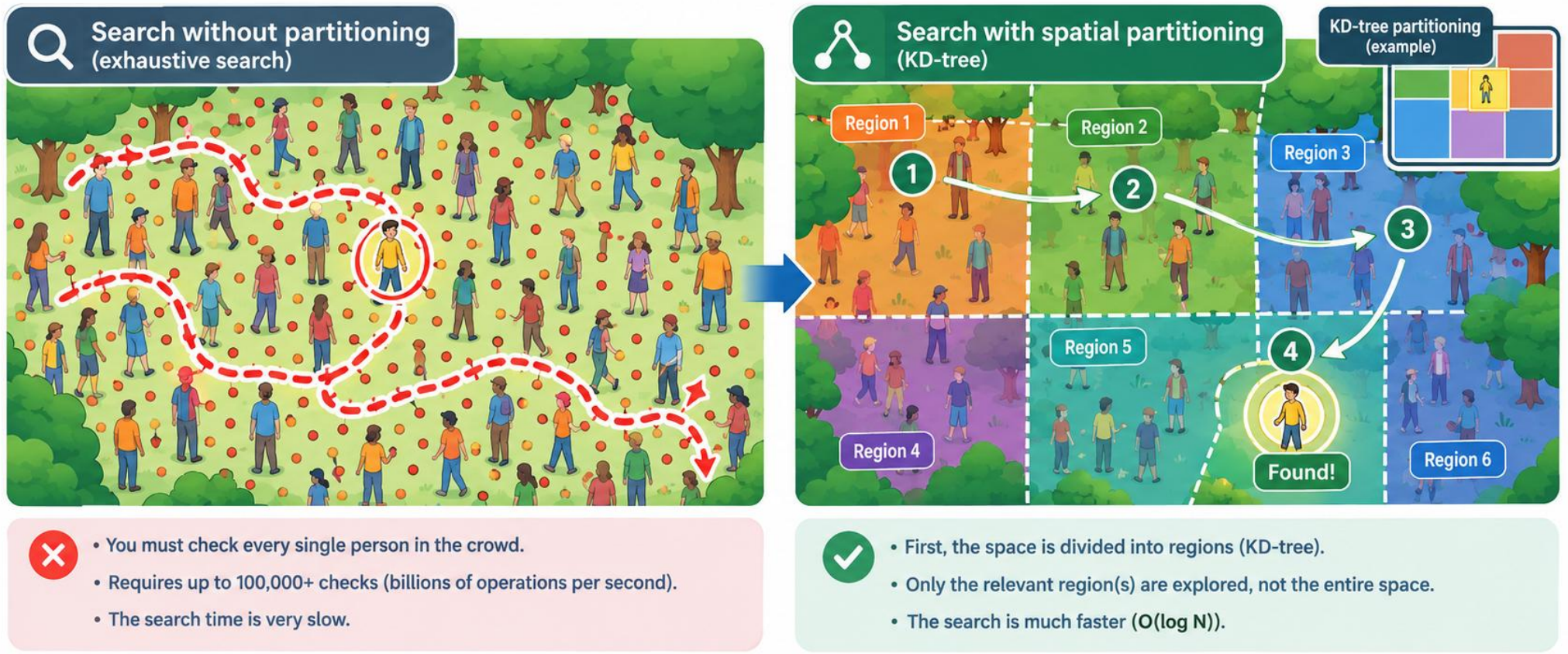



*Immagine generata con AI*

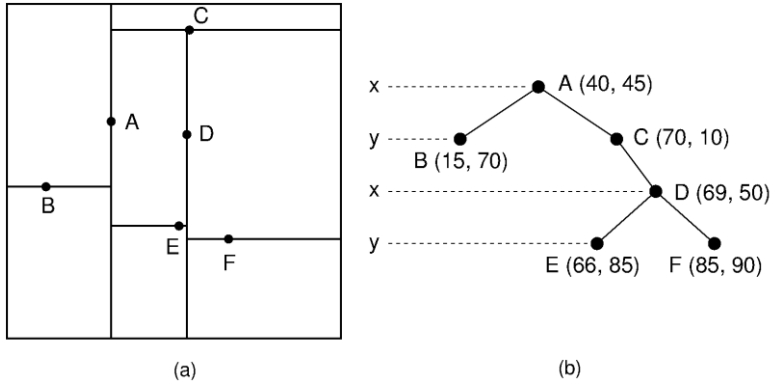
# K-D Tree definition

| *k*-d tree | |
|---|---|
| **Type** | Multidimensional BST |
| **Invented** | 1975 |
| **Invented by** | Jon Louis Bentley |

**Time complexity in big O notation**

| Algorithm | Average | Worst case |
|---|---|---|
| **Space** | $O(n)$ | $O(n)$ |
| **Search** | $O(\log n)$ | $O(n)$ |
| **Insert** | $O(\log n)$ | $O(n)$ |
| **Delete** | $O(\log n)$ | $O(n)$ |

- A K-D Tree is a binary tree in which each node represents a k-dimensional point
- **Every non-leaf node in the tree acts as a hyperplane, dividing the space into two partitions**
- **There are different strategies for choosing an axis when dividing, but the most common one would be to cycle through each of the K dimensions repeatedly and select a midpoint along it to divide the space**
  - 2D example: we first split along the x-axis, then the y-axis, and then the x-axis again, continuing in this manner until all points are accounted for



# K-D Tree construction algorithm

1. **Initialization**:
   - Start by selecting an axis (e.g., *x*-axis)
2. **Median selection**:
   - Sort the points along the selected axis
   - Choose the median point as the splitting hyperplane
3. **Partitioning**:
   - Divide the points into two subsets:
     - Left child node: Points left of the hyperplane
     - Right child node: Points right of the hyperplane
4. **Recursive construction**:
   - Recursively apply for each child node:
     - Select a new axis for each level of the tree
     - Split the space based on the position relative to the new median hyperplane

**Algorithm 1:** K-D Tree Construction

**Function** *kdtree(pointList, depth)*:
**Input:** a list of points, *pointList*, and an integer, *depth* ;
**Output:** a k-d tree rooted at the median point of *pointList* ;

```
/* Select axis based on depth so that axis cycles
   through all valid values                          */
```
**let** $axis := depth \mod k$;

```
/* Sort point list and choose median as pivot element
   */
```
**select** median by *axis* from *pointList*;

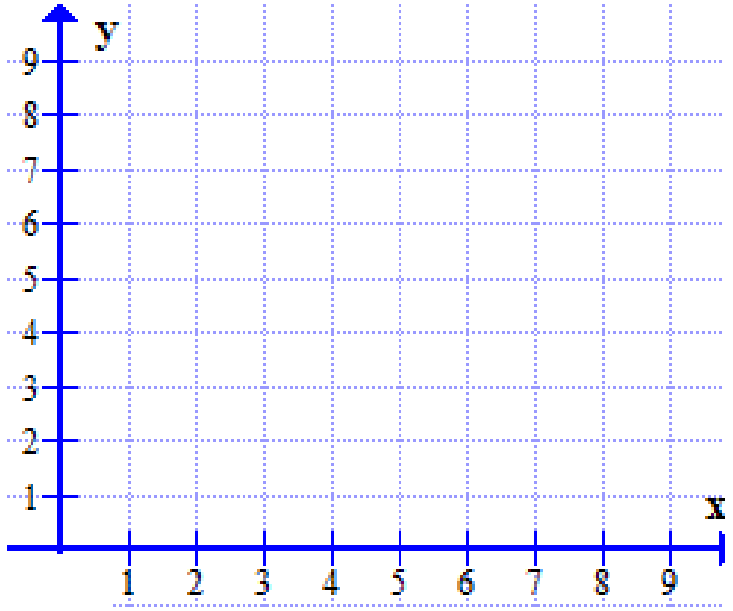
```
/* Create node and construct subtree                 */
```
**let** *node.location* := *median*;
**let** *node.leftChild* := kdtree(points in *pointList* before *median*, $depth + 1$);
**let** *node.rightChild* := kdtree(points in *pointList* after *median*, $depth + 1$);

**return** *node*;

# K-D Tree construction example

[2,3], [8,1], [4,5], [3,9], [1,4], [6,2], [7,7]

Build a 2D K-D tree!




**Function** *kdtree(pointList, depth)*:
**Input:** a list of points, *pointList*, and an integer, *depth* ;
**Output:** a k-d tree rooted at the median point of *pointList* ;

```
/* Select axis based on depth so that axis cycles
   through all valid values                          */
```
**let** *axis* := *depth* mod *k*;

```
/* Sort point list and choose median as pivot element
   */
```
**select** median by *axis* from *pointList*:

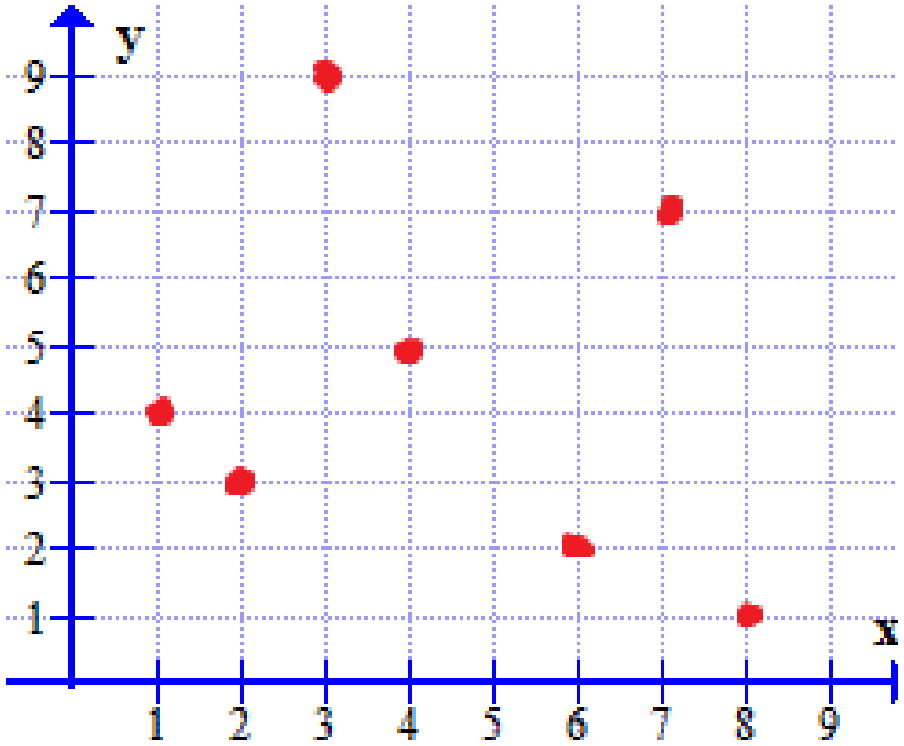
# K-D Tree construction example

[2,3], [8,1], [4,5], [3,9], [1,4], [6,2], [7,7]

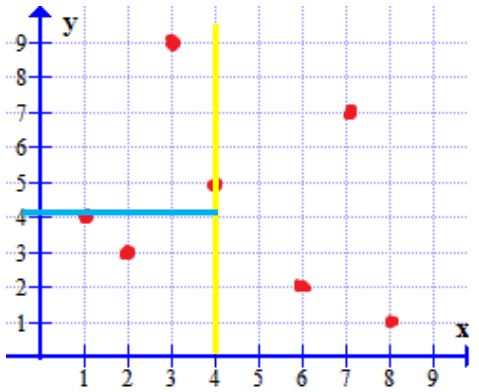
↓

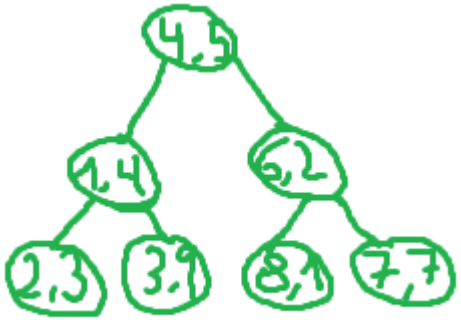
[1,4], [2,3], [3,9], [4,5], [6,2], [7,7], [8,1]

1) Sort the points based on one axis




2) Find the median in $x$ and start partitioning the tree with the median as a root in the tree
3) Repeat iteratively using the median in $y$ then in $x$ and so on…

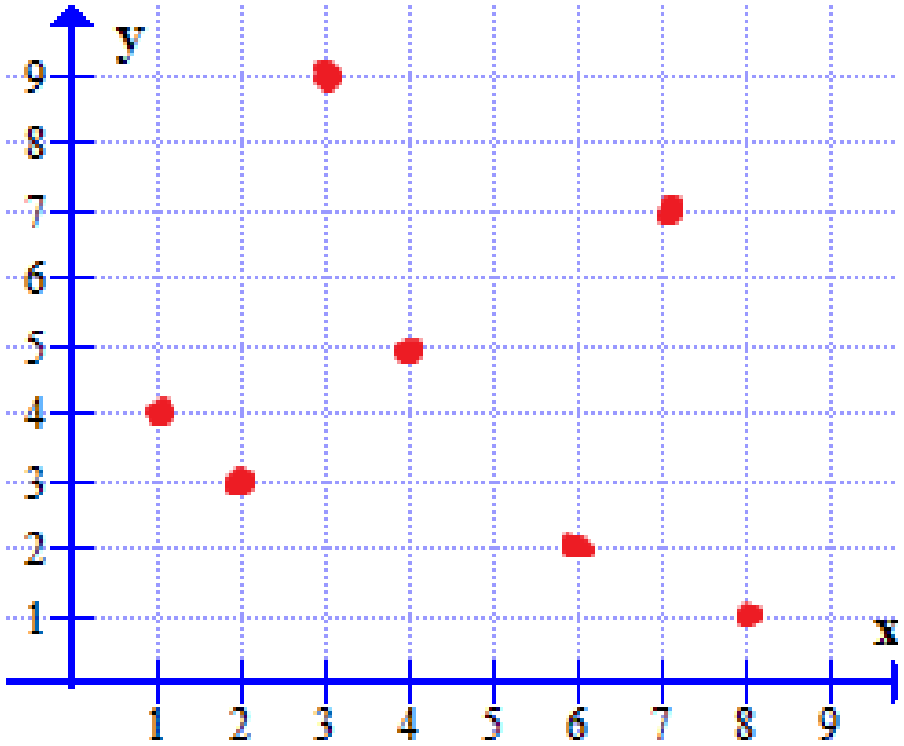


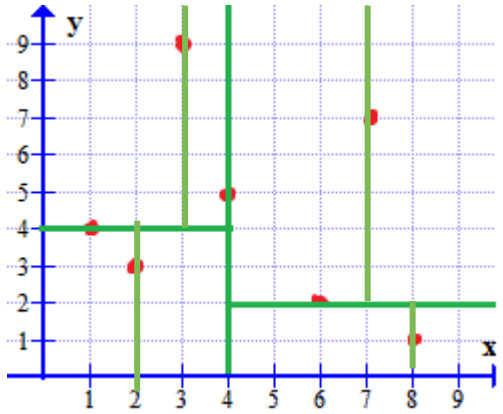
# K-D Tree construction example

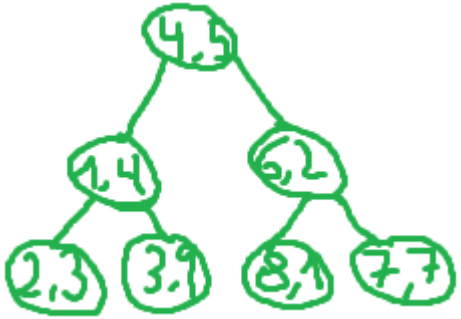
[2,3], [8,1], [4,5], [3,9], [1,4], [6,2], [7,7]

[1,4], [2,3], [3,9], [4,5], [6,2], [7,7], [8,1]




1) Find the median in *x* and start partitioning the tree with the median as a root in the tree
2) Repeat iteratively using the median in *y* then in *x* and so on…



# K-D Tree construction example 2

If the number of elements is odd, the median is found by taking the middle index, (num_elements+1/2)
It depends if we round down or up
In this example we assume rounding up

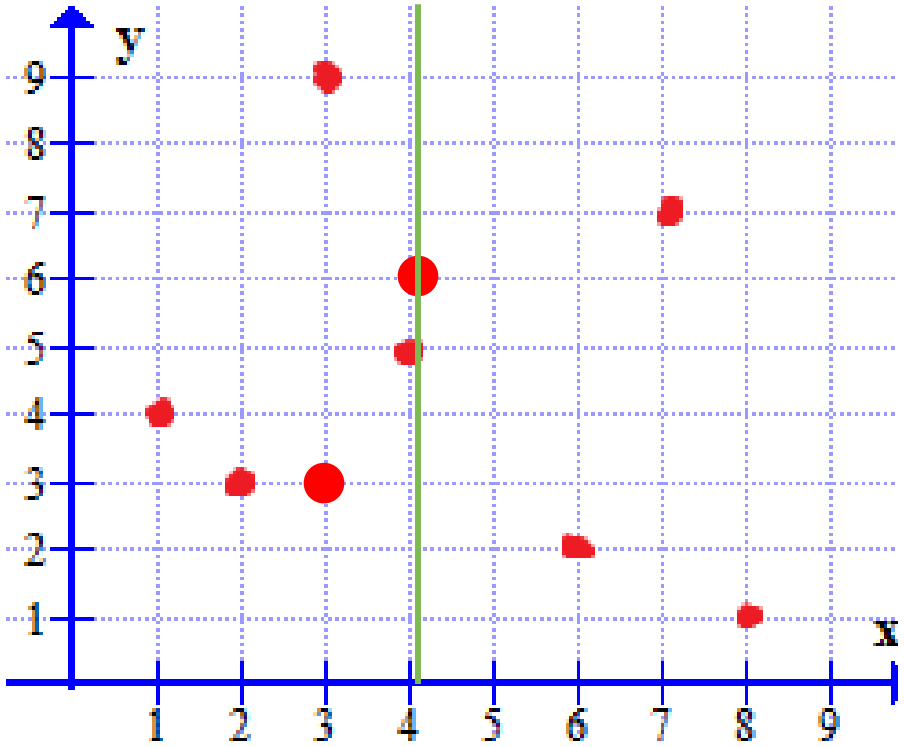
[2,3], [8,1], [4,5], [4,6], [3,3], [3,9], [1,4], [6,2], [7,7]

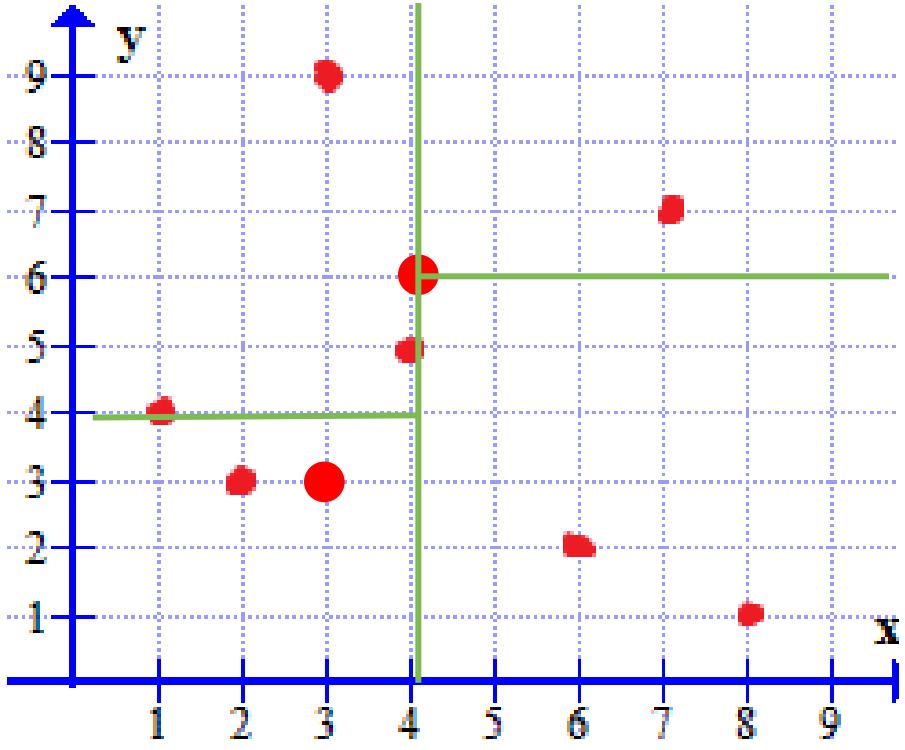
[1,4], [2,3], [3,3], [3,9], [4,5], [4,6], [6,2], [7,7], [8,1]

Sorted by y-coordinate
Find the median in the y coordinate

[2,3], [3,3], **[1,4]**, [3,9]

[8,1], [6,2], **[4,6]**, [7,7]



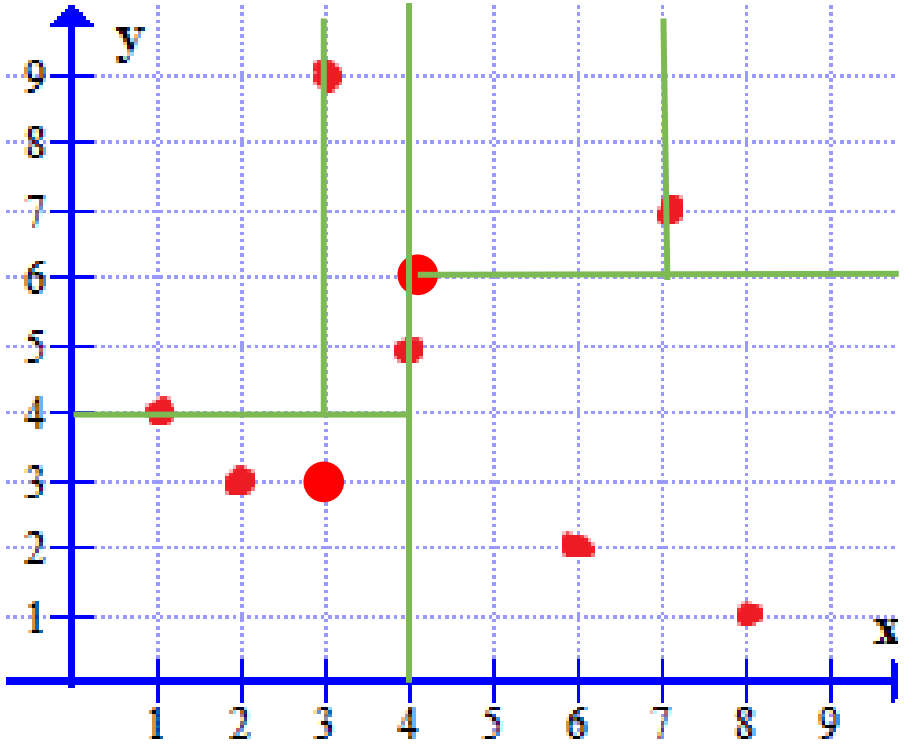
# K-D Tree construction example 2

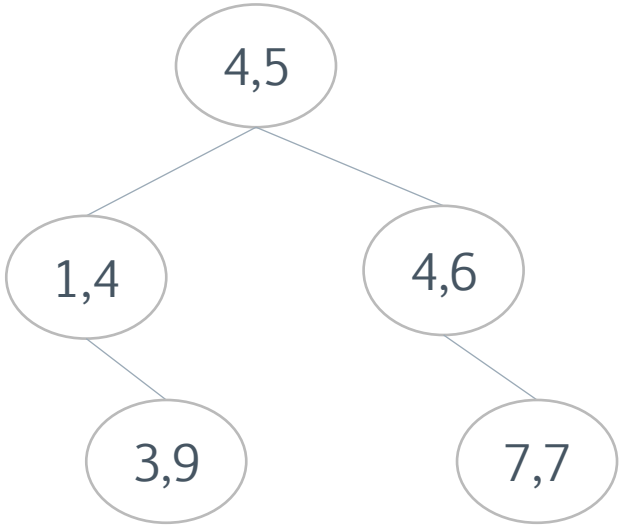
Sorted by y-coordinate
Find the median in the y coordinate

[2,3], [3,3], **[1,4],** [3,9]

[2,3],[3,3] are the left subtree and [3,9] the right

[8,1], [6,2], **[4,6],** [7,7]



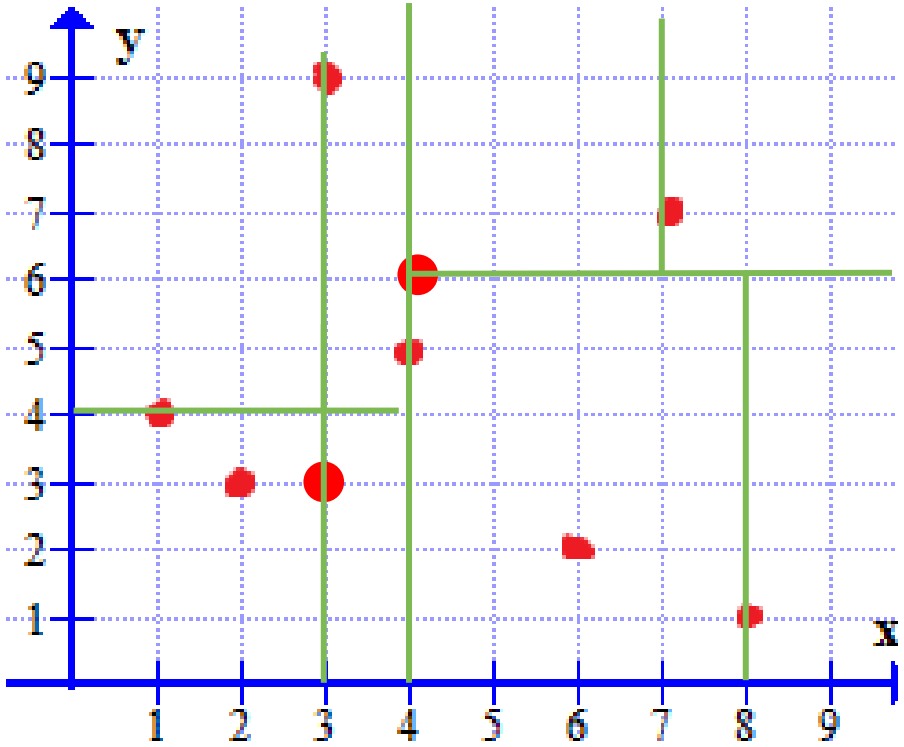



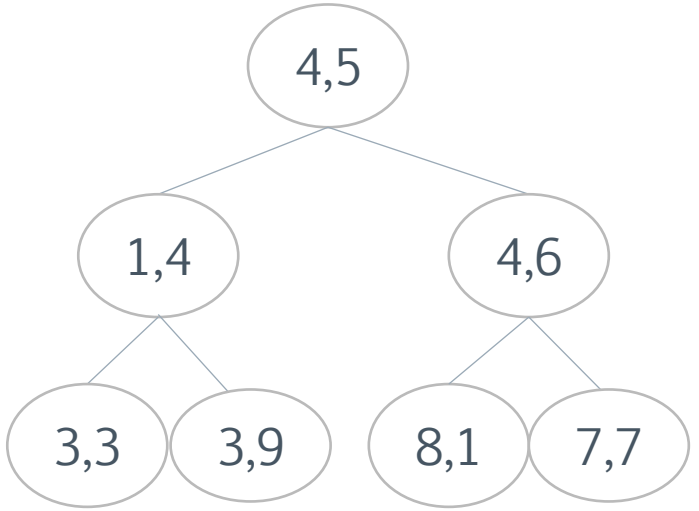
# K-D Tree construction example 2

Sorted by x-coordinate
Find the median in the x coordinate

[2,3], [3,3]

[6,2], [8,1]



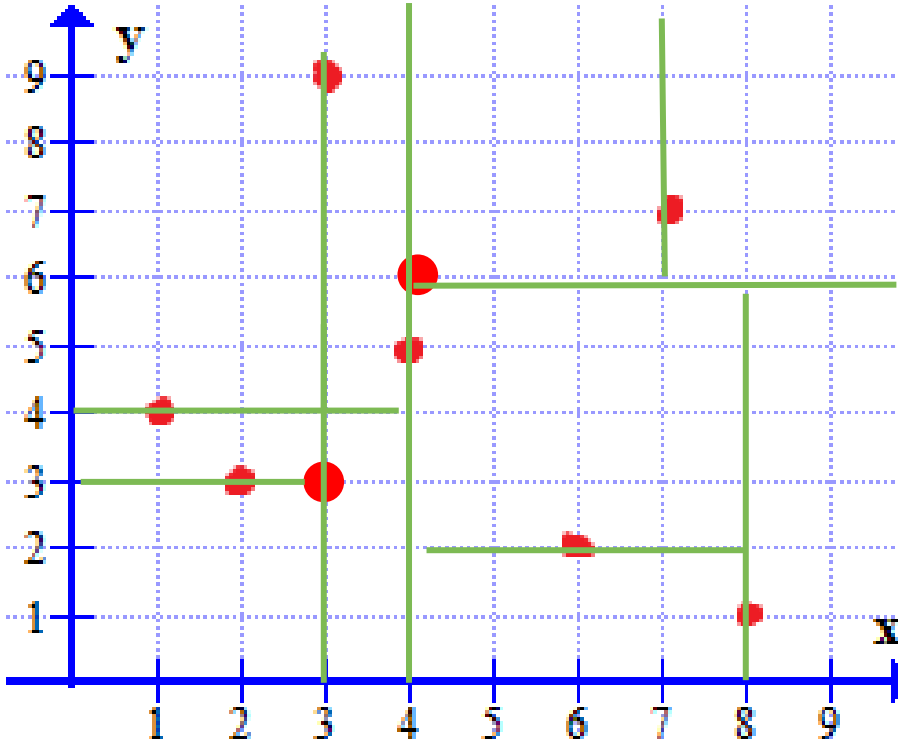



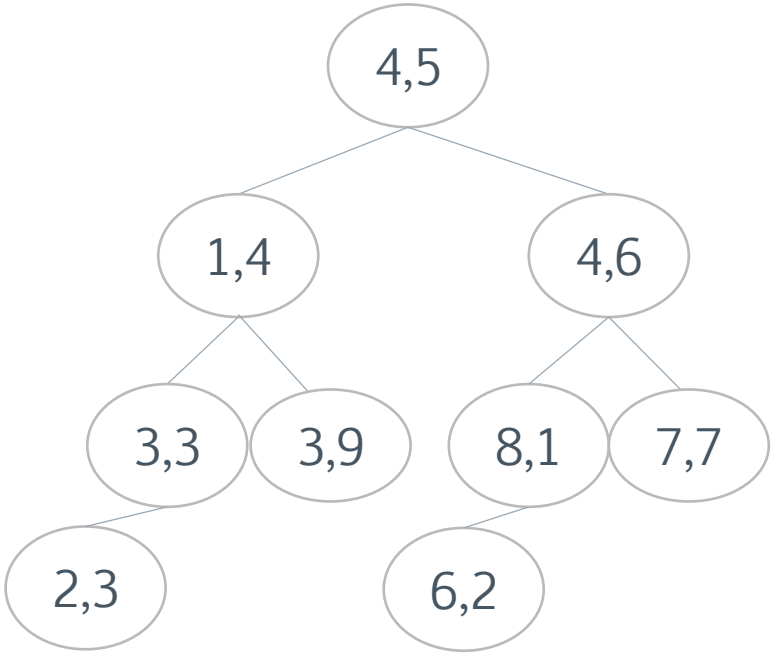
# K-D Tree construction example 2

Sorted by x-coordinate
Find the median in the x coordinate

[2,3],

[6,2],



# K-D Tree search

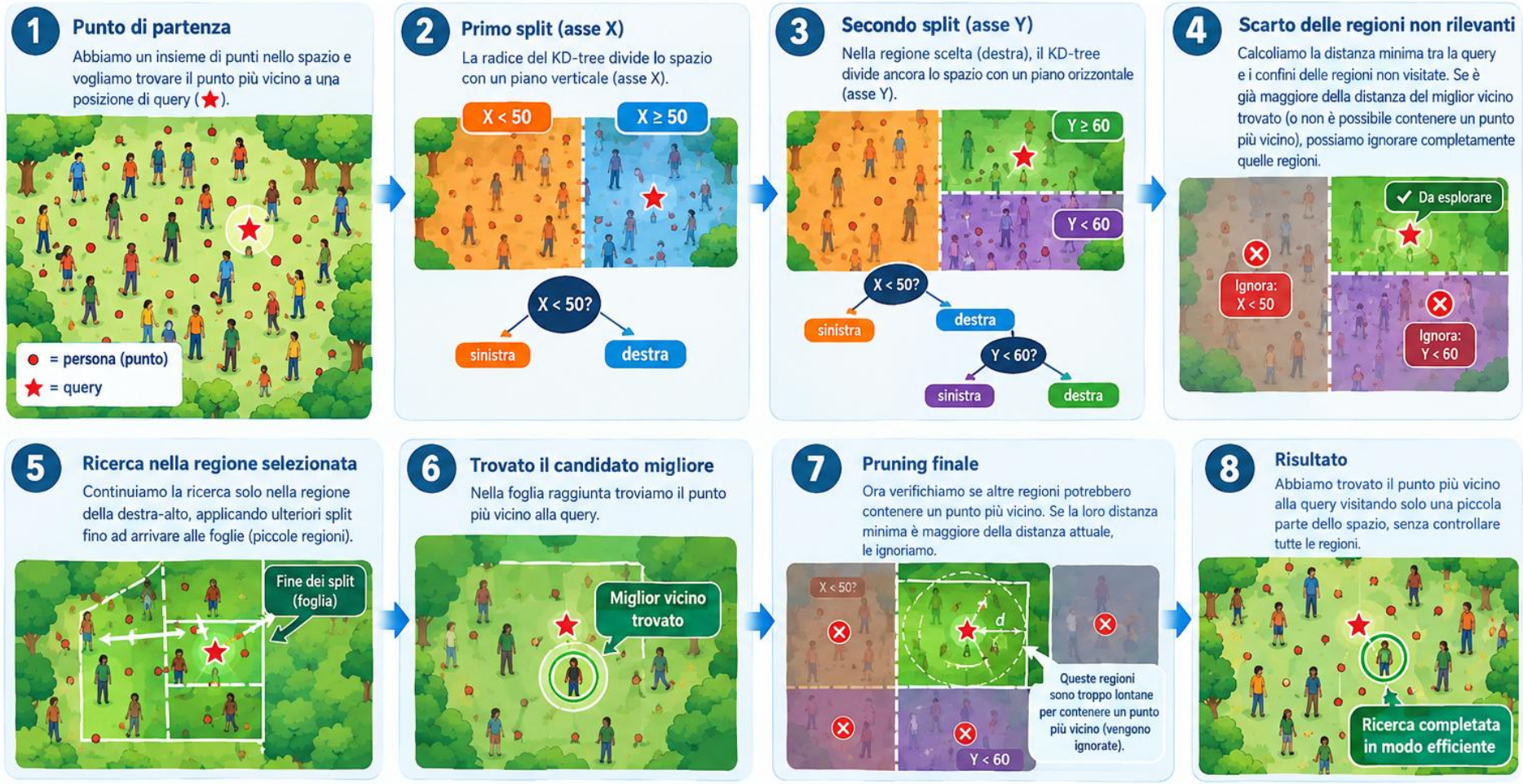
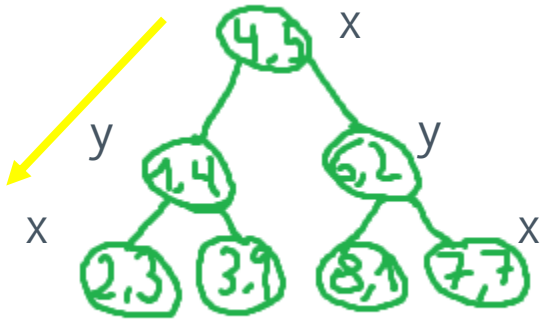


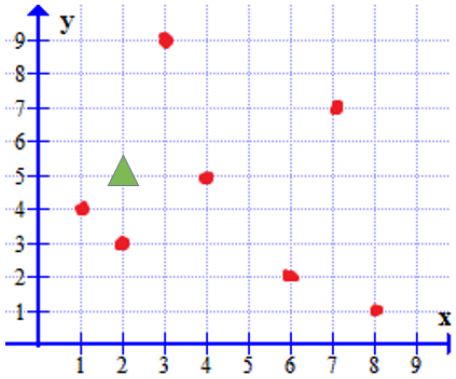

*Immagine generata con AI*

# K-D Tree search example




## Find the closest point to [xs=2, ys=5]

1. Traversal
    1. Begin at the root and recursively traverse the tree
    2. Choose the left or right subtree based on the query point's position relative to the splitting hyperplane
        1. First node: is (xs < x) where [x=4, y=5]; yes, explore left branch; no, explore right branch
        2. Second node: is (ys < y) where [x=1, y=4]; yes, explore left branch; no, explore right branch
    3. We compute the distance while exploring and check the score
2. Leaf node comparison
    1. Reach a leaf node and compute the distance to the query point
    2. Check the distance and store it if it is the "current best" distance
3. Recursion unwinding
    1. As you move back up the tree, keep updating the "current best" if a closer point is found
    2. Check for potential closer points across the splitting plane
4. Hypersphere intersection
    1. Determine if a hypersphere centered at the query point intersects the splitting hyperplane
    2. This check involves comparing the absolute distance along the axis with the distance to the "current best"
5. Branch decision:
    1. If the hypersphere intersects, explore the opposite subtree for potentially closer points
    2. If not, ignore the opposite branch and continue unwinding
6. End of search:
    1. The search concludes once the root node is processed, confirming the nearest neighbor
7. Distance metric:
    1. Typically uses Euclidean distance:
    2. Adaptable to other metrics, changing the geometry of the intersection test





# K-D Tree search

```
Node nearestNeighbor(Node root, Point target, int depth) {

    if (root == null) return null

    if (target[depth % K] < root.point[depth % K]) {
        nextBranch = root.left
        otherBranch = root.right
    } else {
        nextBranch = root.right
        otherBranch = root.left
    }

    Node temp = nearestNeighbor(nextBranch, target, depth + 1)
    Node best = closest(target, temp, root)

    long radiusSquared = distSquared(target, best.point)
    long dist = target[depth % K] - root.point[depth % K]

    if (radiusSquared >= dist * dist) {
        temp = nearestNeighbor(otherBranch, target, depth + 1)
        best = closest(target, temp, best)
    }

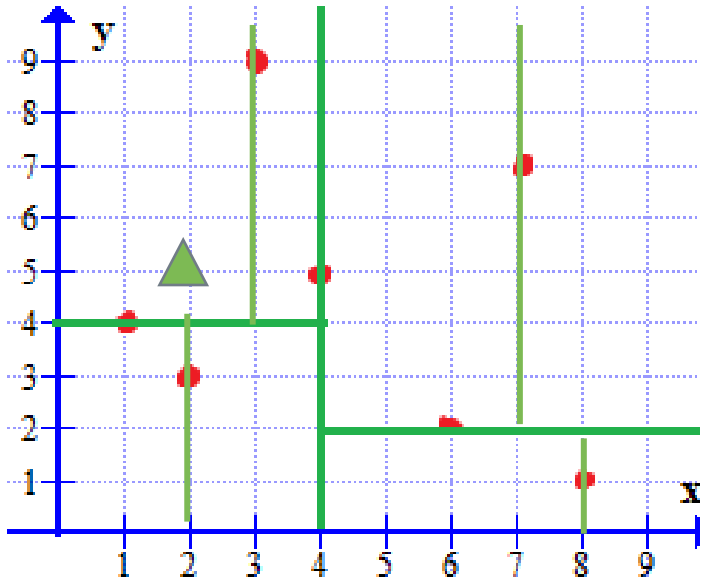
    return best
}
```

Find the closest point to [2,5]

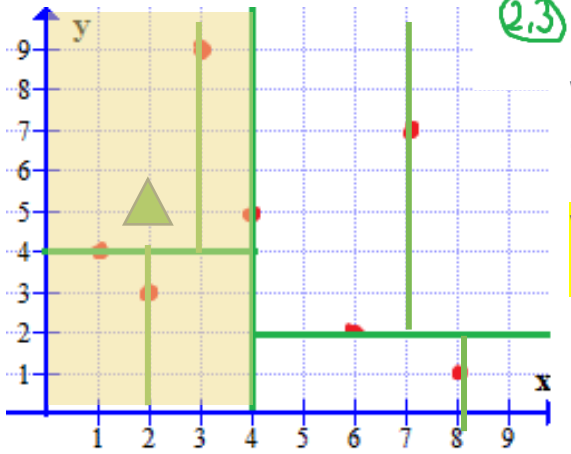
# K-D Tree search example

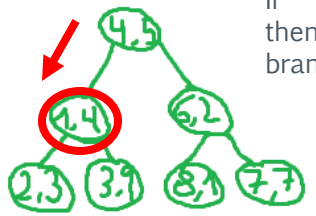
0

We save the euclidean distance between [2,5] and [4,5] which is 2




1



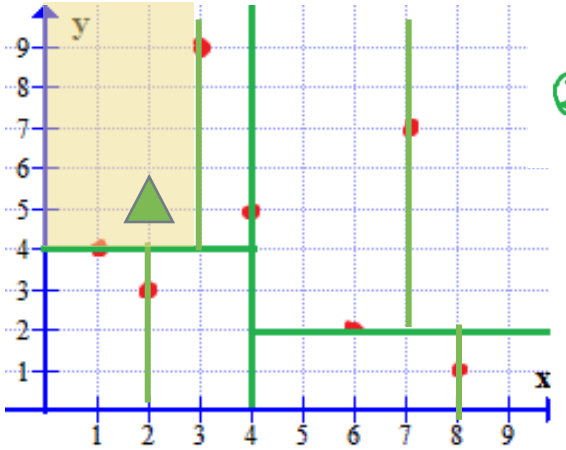



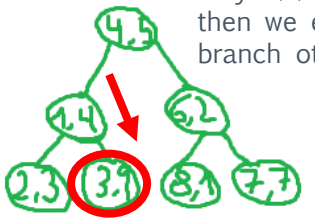

If " x" (2) is greater than 4 then we explore the right branch otherwise left branch

We evaluate the euclidean distance between [2,5] and [1,4]

We save the distance which is 1.41

2




If "y" (5) is greater than 4 then we explore the right branch otherwise left branch

We save the euclidean distance which is 4.12



# K-D Tree backtracking example

1) It is possible that exploring the tree we do not find the closest point
2) We need to explore also the other branches if the sphere with the radius intersects with other regions
   - Unwind the recursion and check if it is the closest point
   - **RULE: if the perpendicular distance between the point and the other region (r') is less than the distance found (r) – *that now is 1.41*, then we can use that information to unwind the other branch**
3) Repeat iteratively until finding the closest point

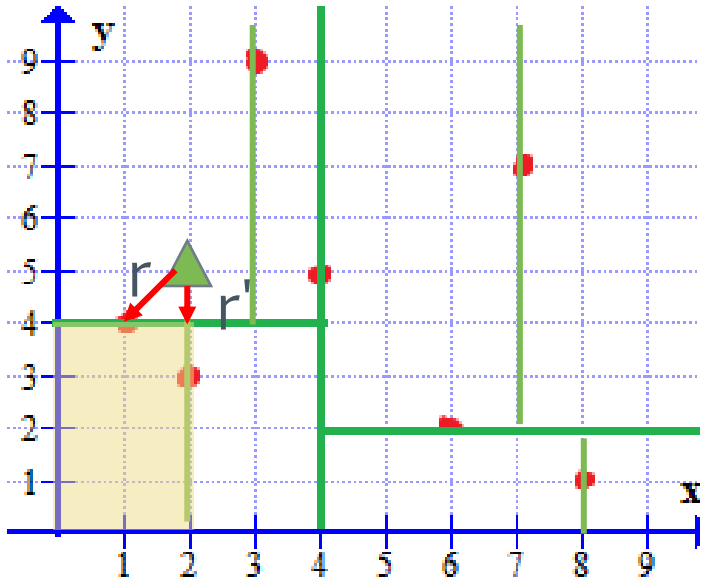
# K-D Tree search example




3

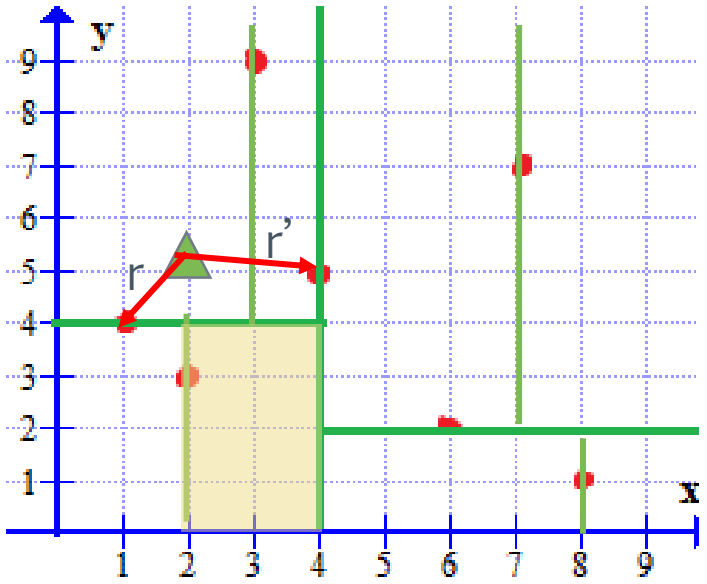
We do backtrack because r > r' We explore the left subtree of [1,4] which is [2,3]

Distance with [2,3] is 2




4

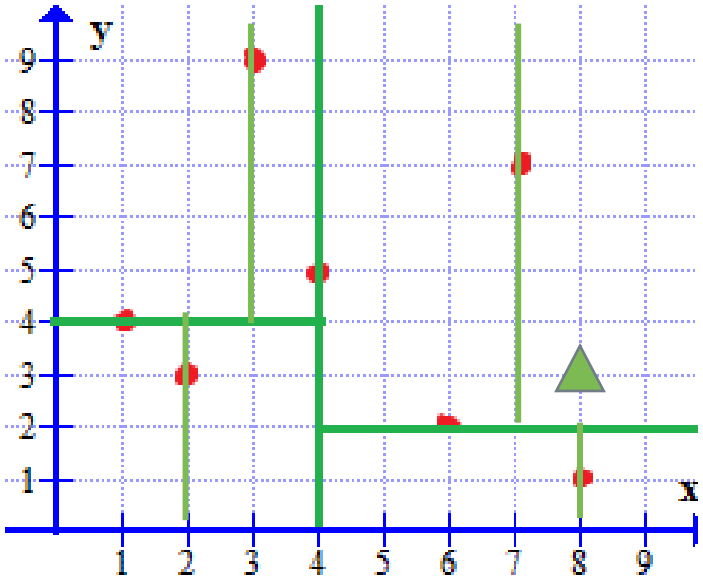
We have already explored the left subtree. Since the distance to the root separation plane r' (coordinate x=4x=4) is less than the best distance "r" (the distance is 2, which is equal to the best distance found so far), there is no need to explore the right subtree further (stop condition)

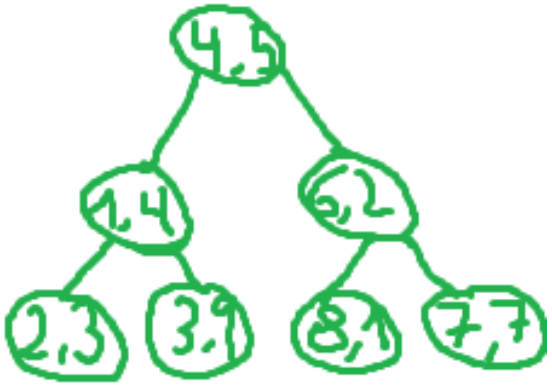



**Backtrack su (1,4)**: distanza dal piano Y=4 è |5−4| = **1** < r (r è 1.41)
⇒ esplora anche il lato opposto → **(2,3)**.
•dist(Q,(2,3)) = 2 → best rimane 2 (pari al migliore attuale)

# K-D Tree search example

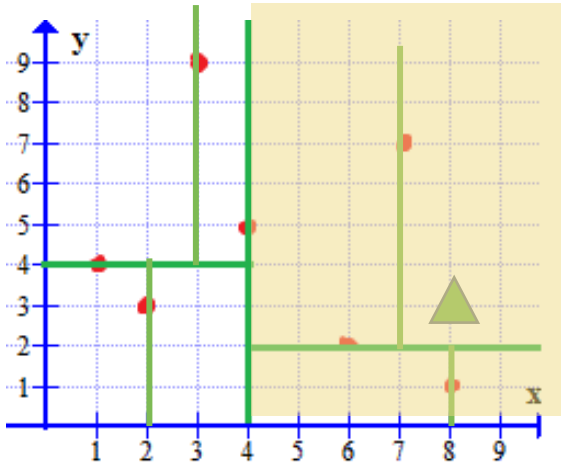
Find the nearest neighbor of the point [8,3]



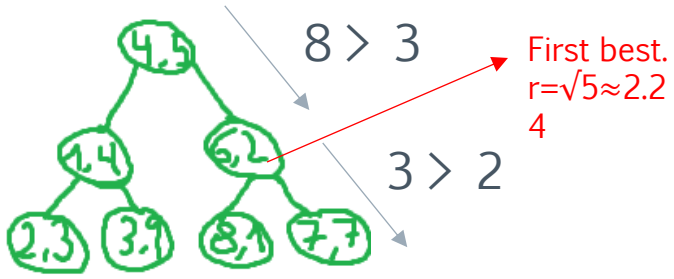



# K-D Tree search example

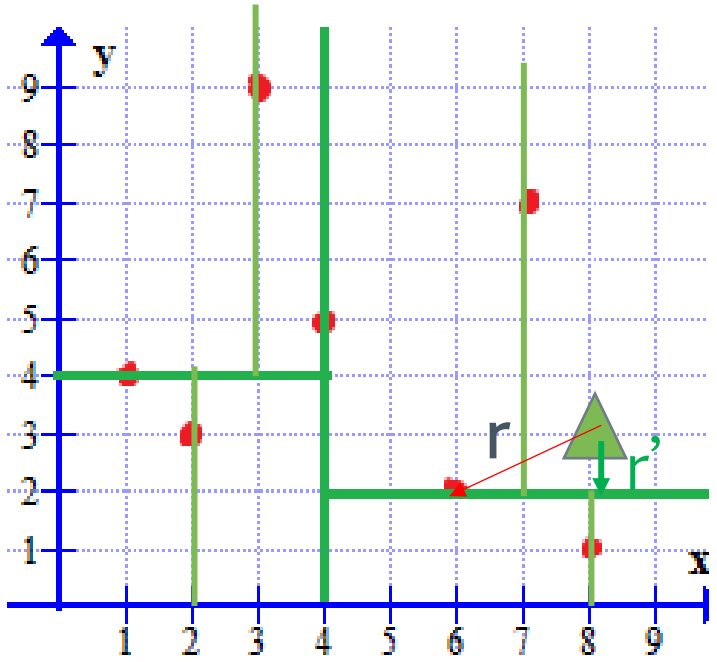
1




Find the nearest neighbor of the point [8,3]

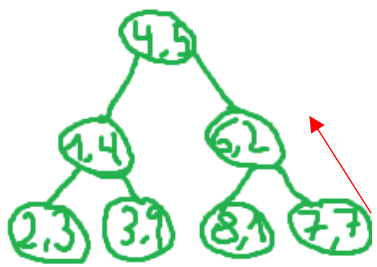


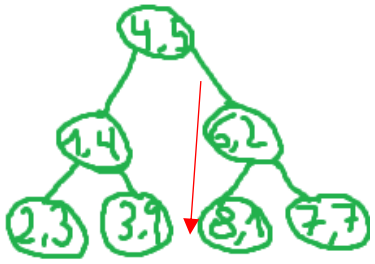

2




If the line directly perpendicular to the section that we chose not to visit when recursing down, is shorter, we recurse back

We have explored the right subtree of (4,5) so there is no need to explore the left subtree further as the distance to the separation plane x=4x=4 is greater than the best_distance





# PCL KDTree

- The pcl_kdtree library provides the kd-tree data-structure that allows for fast nearest neighbor searches
- pcl::search::KdTree<pcl::PointXYZ>::Ptr tree (new pcl::search::KdTree<pcl::PointXYZ>);
  - setInputCloud
    - Provide a pointer to the input dataset.
  - **This object is later used to define the method use for searching nearest neighbors**

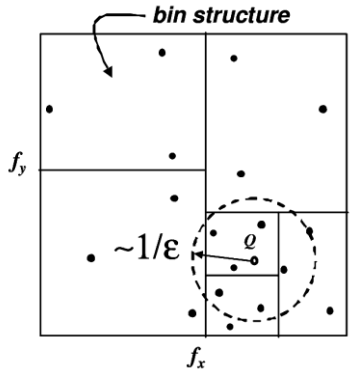
# Clustering

- We need a method to establish the points belonging to a certain cluster
- **Euclidean clustering**: we use a K-D tree structure for finding the nearest neighbors
  - To define: Euclidean distance and **a search threshold**
    - Straight line distance between two points

$$searchpoint(x) + threshold > x > searchpoint(x) - threshold$$
$$searchpoint(y) + threshold > y > searchpoint(y) - threshold$$
$$searchpoint(z) + threshold > z > searchpoint(z) - threshold$$

$$Distance_{(searchpoint\ ,\ currentpoint\ )} = \sqrt{(x - searchpoint(x))^2 + (y - searchpoint(y))^2 + (z - searchpoint(z))^2}$$

# Clustering




- We use the Euclidean distance and the KD-tree to determine clusters
- **A search radius is defined**
  - *setClusterTolerance* **is the radius for the k-NN searching, used to find the candidate points**

```
// Creating the KdTree object for the search method of the extraction
pcl::search::KdTree<pcl::PointXYZ>::Ptr tree (new pcl::search::KdTree<pcl::PointXYZ>);
tree->setInputCloud (cloud_filtered);

// Here we are creating a vector of PointIndices, which contain the actual index information in a vector<int>. The indices of each detected
cluster are saved here.
std::vector<pcl::PointIndices> cluster_indices;
pcl::EuclideanClusterExtraction<pcl::PointXYZ> ec;

//Set the spatial tolerance for new cluster candidates
//If you take a very small value, it can happen that an actual object can be seen as multiple clusters. On the other hand, if you set the
value too high, it could happen, that multiple objects are seen as one cluster
ec.setClusterTolerance (0.02); // 2cm

//We impose that the clusters found must have at least setMinClusterSize() points and maximum setMaxClusterSize() points
ec.setMinClusterSize (100);
ec.setMaxClusterSize (25000);
ec.setSearchMethod (tree);
ec.setInputCloud (cloud_filtered);
ec.extract (cluster_indices);
```

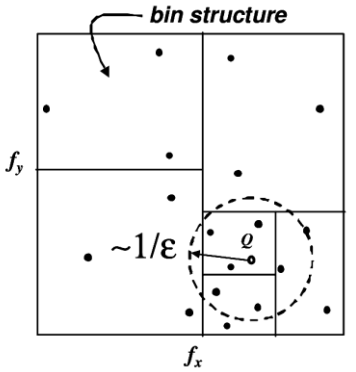
# PCL Euclidean clustering

- Here we are creating a EuclideanClusterExtraction object with point type PointXYZ since our point cloud is of type PointXYZ
- setClusterTolerance()
  - If you take a very small value, it can happen that an actual object can be seen as multiple clusters. If you set the value too high, multiple objects can be seen as one cluster
- We impose that the clusters found must have at least setMinClusterSize() points and maximum setMaxClusterSize() points
- Now we extracted the clusters out of our point cloud and saved the indices in cluster_indices. To separate each cluster out of the vector<PointIndices> we have to iterate through cluster_indices, create a new PointCloud for each entry and write all points of the current cluster in the PointCloud

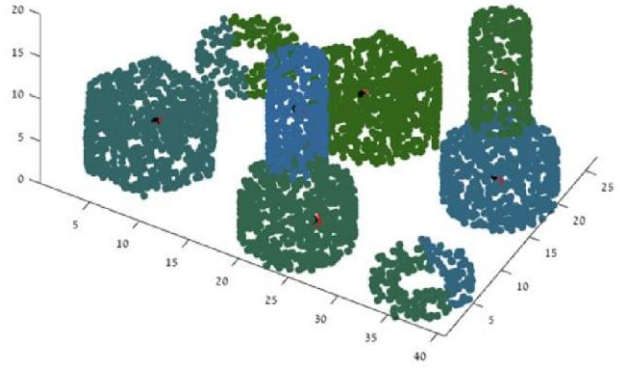
```
pcl::EuclideanClusterExtraction<pcl::PointXYZ> ec;
ec.setClusterTolerance (0.02); // 2cm
ec.setMinClusterSize (100);
ec.setMaxClusterSize (25000);
ec.setSearchMethod (tree);
ec.setInputCloud (cloud_filtered);
ec.extract (cluster_indices);
```

```
for (const auto& cluster : cluster_indices)
{
    pcl::PointCloud<pcl::PointXYZ>::Ptr cloud_cluster
(new    pcl::PointCloud<pcl::PointXYZ>);
    for (const auto& idx : cluster.indices) {
      cloud_cluster->push_back((*cloud_filtered)[idx]);
    } //*
    cloud_cluster->width = cloud_cluster->size ();
    cloud_cluster->height = 1;
    cloud_cluster->is_dense = true;
}
```

# K-D tree nearest neighbors

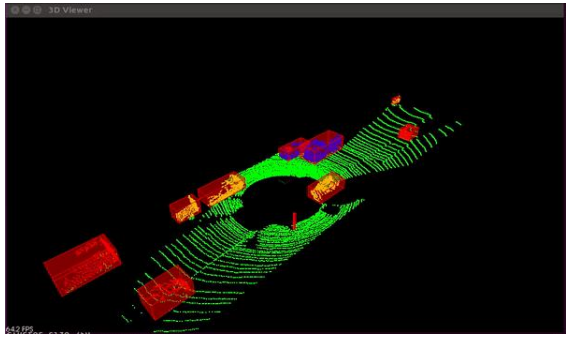




1. create a kd-tree representation for the input point cloud dataset $\mathcal{P}$;
2. set up an empty list of clusters $C$, and a queue of the points that need to be checked $Q$;
3. then for every point $\boldsymbol{p}_i \in \mathcal{P}$, perform the following steps:
   - add $\boldsymbol{p}_i$ to the current queue $Q$;
   - for every point $\boldsymbol{p}_i \in Q$ do:
     - search for the set $\mathcal{P}_i^k$ of point neighbors of $\boldsymbol{p}_i$ in a sphere with radius $r < d_{th}$;
     - for every neighbor $\boldsymbol{p}_i^k \in \mathcal{P}_i^k$, check if the point has already been processed, and if not add it to $Q$;
   - when the list of all points in $Q$ has been processed, add $Q$ to the list of clusters $C$, and reset $Q$ to an empty list
4. the algorithm terminates when all points $\boldsymbol{p}_i \in \mathcal{P}$ have been processed and are now part of the list of point clusters $C$.

# Euclidean clustering detection

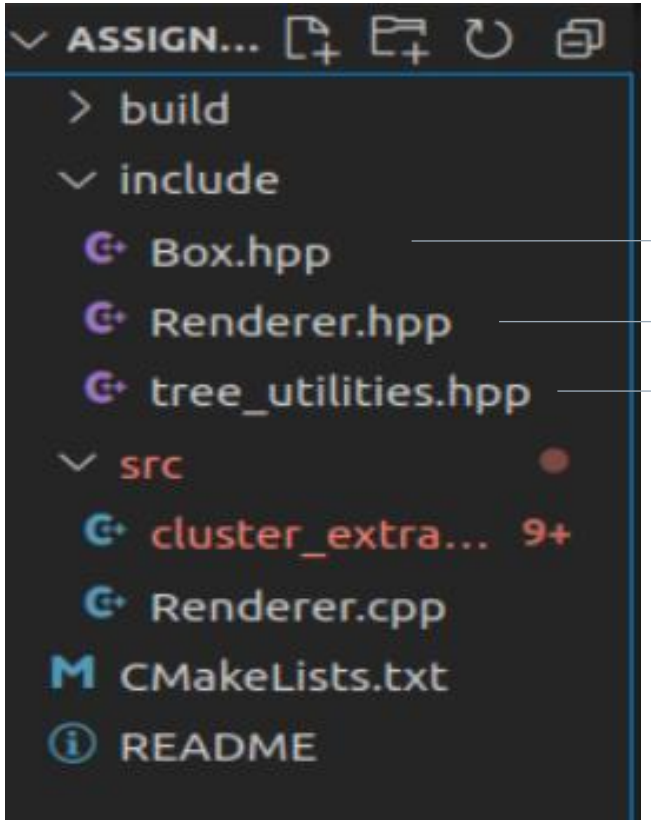


› **Objective**: Find and segment the individual object point clusters lying on the plane

1. Down sample the point cloud
2. Segment the ground plane
3. Create a KD-tree representation for the input point cloud dataset
4. Compute Euclidean distance to build the cluster list
5. The algorithm terminates when all points have been processed and are now part of the list of point clusters

```
sudo apt-get install pcl-tools
sudo apt install libpcl-dev
```

# Euclidean clustering detection




Bounding Box struct definition

Renderer class definition

Functions and data structure for defining a kdtree

```
namespace lidar_obstacle_detection
{

  struct BoxQ
  {
    Eigen::Vector3f bbox_transform;
    Eigen::Quaternionf bbox_quaternion;
    float cube_length;
    float cube_width;
    float cube_height;
  };

  struct Box
  {
    float x_min;
    float y_min;
    float z_min;
    float x_max;
    float y_max;
    float z_max;
  };

}
```

```
enum class CameraAngle
{
  XY, TopDown, Side, FPS
};

class Renderer
{
private:
  pcl::visualization::PCLVisualizer::Ptr viewer_;
  unsigned long long rays_counter_;

public:
  Renderer();

  void RenderHighway();

  void RenderRays(const Eigen::Vector3f& origin, const pcl::PointCloud<pcl::PointXYZ>::Ptr& cloud);

  void ClearRays();

  void ClearViewer();

  void RenderPointCloud(const pcl::PointCloud<pcl::PointXYZ>::Ptr& cloud,
                        const std::string& name,
                        const Color& color = Color(1,1,1));

  void RenderPointCloud(const pcl::PointCloud<pcl::PointXYZI>::Ptr& cloud,
                        const std::string& name,
                        const Color& color = Color(-1,-1,-1));

  void RenderBox(const Box& box, int id, const Color& color = Color(1,0,0), float opacity = 1.0);

  void RenderBox(const BoxQ& box, int id, const Color& color = Color(1,0,0), float opacity = 1.0);

  void InitCamera(CameraAngle view_angle);

  bool WasViewerStopped() const;

  void SpinViewerOnce() const;

};
```

```
namespace my_pcl
{
// Structure to represent node of kd tree
struct Node
{
    std::vector<float> point;
    int id;
    Node* left;
    Node* right;

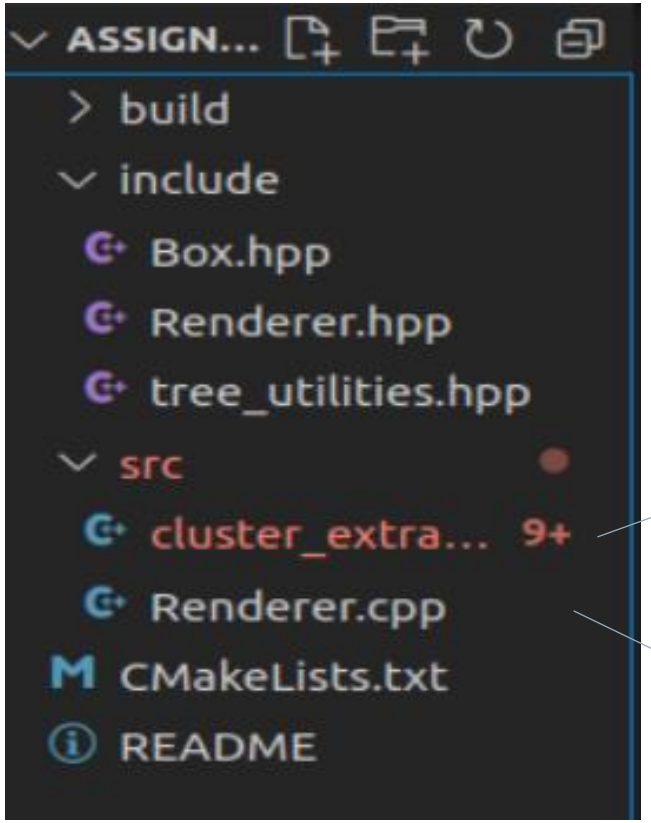
    Node(std::vector<float> arr, int setId)
    :   point(arr), id(setId), left(NULL), right(NULL)
    {}
};


struct KdTree
{
    Node* root;
    int dimension;
    KdTree()
    : root(NULL)
    {
        dimension = 2;
    }

    void set_dimension(int new_dimension)
    {
        dimension = new_dimension;
    }

    int get_split_by_index(int depth)
    {
        return (depth % dimension);
    }
```

# Euclidean clustering detection




```
//This function sets up the custom kdtree using the point cloud
void setupKdtree(typename pcl::PointCloud<pcl::PointXYZ>::Ptr cloud, my_pcl::KdTree* tree, int dimension)
```

```
/*
OPTIONAL
This function computes the nearest neighbors and builds the clusters
    - Input:
        + cloud: Point cloud to be explored
        + target_ndx: i-th point to visit
        + tree: kd tree for searching neighbors
        + distanceTol: Distance tolerance to build the clusters
        + visited: Visited points --> typedef std::unordered_set<int> my_visited_set_t;
        + cluster: Here we add points that will represent the cluster
        + max: Max cluster size
    - Output:
        + visited: already visited points
        + cluster: at the end of this function we will have one cluster
*/
void proximity(typename pcl::PointCloud<pcl::PointXYZ>::Ptr cloud, int target_ndx, my_pcl::KdTree* tree, float distanceTol, my_visited_set_t&
```

```
/*
OPTIONAL
This function builds the clusters following a euclidean clustering approach
    - Input:
        + cloud: Point cloud to be explored
        + tree: kd tree for searching neighbors
        + distanceTol: Distance tolerance to build the clusters
        + setMinClusterSize: Minimum cluster size
        + setMaxClusterSize: Max cluster size
    - Output:
        + cluster: at the end of this function we will have a set of clusters
TODO: Complete the function
*/
std::vector<pcl::PointIndices> euclideanCluster(typename pcl::PointCloud<pcl::PointXYZ>::Ptr cloud, my_pcl::KdTree* tree, float distanceTol,
```

```
void
ProcessAndRenderPointCloud (Renderer& renderer, pcl::PointCloud<pcl::PointXYZ>::Ptr& cloud)
```

```
int main(int argc, char* argv[])
```

Functions for render the point cloud and other render objects



# Euclidean clustering detection

```
// 8) Set up the euclidean cluster parameters
#ifdef USE_PCL_LIBRARY

  //PCL functions

#else

  my_pcl::KdTree treeM;
  treeM.set_dimension(3);
  setupKdtree(cloud_filtered, &treeM, 3);
  //cluster_indices = euclideanCluster(cloud_fi
#endif
```

- **Objective**: Find and segment the individual object point clusters lying on the plane (15 points)
  1. Implement the object clustering using the PCL library
     - The code compiles and works (2 points)
     - Good cluster detection without false positives and high frame rate (5 points)
  2. Implement your own EuclideanClustering algorithm (1 points)
  3. Try to make your solution work on dataset 2. If you cannot successfully provide a solution. Write a short report explaining why (2 point)
  4. Extend the code to support any additional useful functionality (5 points). Examples:
     1. Plot the distance of each cluster w.r.t ego vehicle
     2. Color in red the vehicles that are both in front of and 5 meters away from the ego vehicle
     3. Any other functionality is up to your discretion (use other strategies to create the clustering rather than the Euclidean heuristic)

# Optional (random ideas)

- **Distance-Adaptive / Adaptive-Threshold Clustering:**
  - *Problema:* La densità dei punti LiDAR diminuisce drasticamente all'aumentare della distanza dal sensore. Una tolleranza fissa (es. 0.2m) fonde gli oggetti vicini (under-segmentation) o frammenta i veicoli lontani (over-segmentation).
  - *Task:* Implementare un algoritmo di clustering adattivo in cui il raggio di tolleranza $r(d)$aumenta in funzione della distanza $d$dall'auto (*ego-vehicle*) (es. $r(d) = r_0 + \alpha \cdot d$).
- **Definizione di una Region of Interest (ROI) Dinamica / Risk Assessment:**
  - *Task:* Calcolare il **Time-To-Collision (TTC)** per gli oggetti rilevati sulla traiettoria stimata del veicolo (*Ego Path*) ed evidenziare visivamente a schermo (es. in rosso lampeggiante) solo gli ostacoli che rappresentano un pericolo imminente di collisione.